

Clase 2 - Temario

- Control de Versiones
- Git/GitHub



CONTROL DE VERSIONES

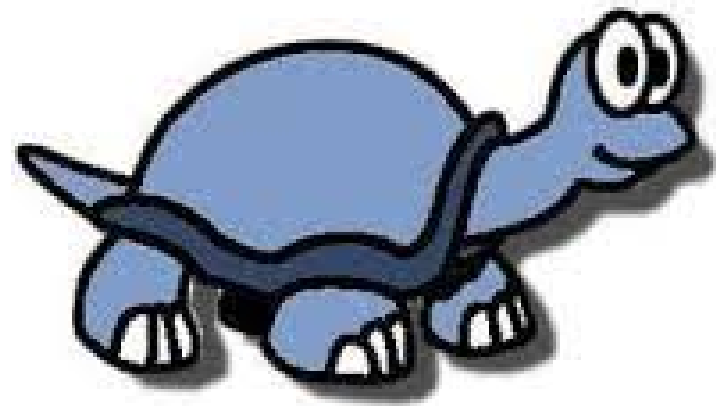


SISTEMA DE CONTROL DE CAMBIOS

ES UNA HERRAMIENTA O SOFTWARE QUE PERMITE GESTIONAR LOS CAMBIOS REALIZADOS EN ARCHIVOS A LO LARGO DEL TIEMPO.

PERMITE A LOS USUARIOS GUARDAR DIFERENTES VERSIONES DE LOS ARCHIVOS, REGISTRAR QUIÉN HIZO QUÉ CAMBIOS Y CUÁNDO, Y COMPARAR Y REVERTIR VERSIONES SI ES NECESARIO.

SISTEMA DE CONTROL DE CAMBIOS



TortoiseSVN



git



Bazaar



HelixCore

SISTEMAS DE CAMBIOS TRADICIONALES

- Se guardan solo los cambios que se realizaron



GIT

GIT ES UN SOFTWARE DE CONTROL DE VERSIONES DISEÑADO POR LINUS TORVALDS.

FUE PENSADO PARA EL MANTENIMIENTO DE VERSIONES DE APLICACIONES - CUANTO TIENEN UN GRAN NÚMERO DE ARCHIVOS DE CÓDIGO FUENTE.

SU PROPÓSITO ES LLEVAR REGISTRO DE LOS CAMBIOS EN ARCHIVOS Y COORDINAR EL TRABAJO DE VARIAS PERSONAS EN ARCHIVOS COMPARTIDOS



UADE

DESARROLLO COLABORATIVO

Proporciona herramientas para que múltiples personas puedan hacer desarrollos en conjunto de una contribuyendo simultáneamente al mismo proyecto.



UADE

GIT VS OTROS SISTEMAS DE CONTROL DE VERSIONES

La principal diferencia entre Git y otros sistemas de control de versiones (SCV) tradicionales se encuentra en cómo almacenan y gestionan la historia de los cambios en un proyecto.

UADE

ENFOQUE TRADICIONAL

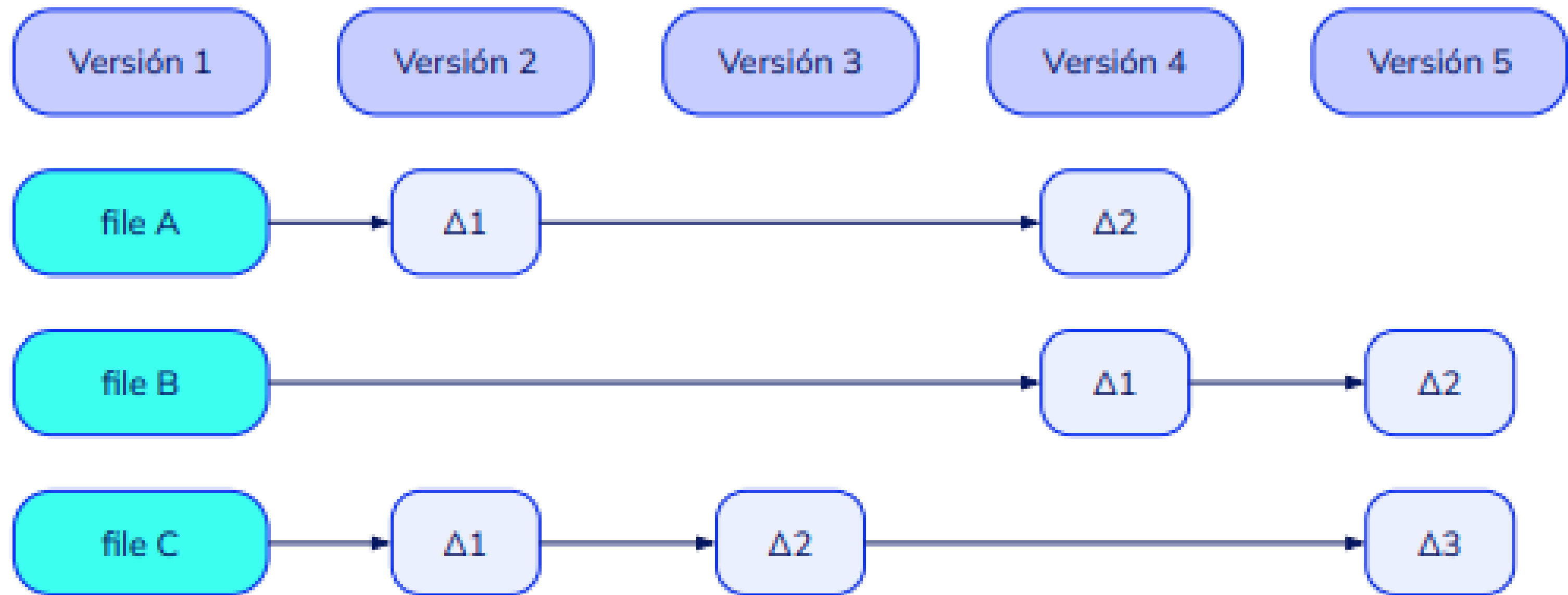
Almacenan la información del proyecto como una serie de diferencias o cambios entre versiones.

- Lista de cambios (Deltas):
 - Estos sistemas guardan la historia de un archivo registrando las diferencias (deltas) entre una versión y la siguiente.

ENFOQUE GIT

Git piensa en sus datos como un conjunto de **snapshots (instantáneas)** de un mini sistema de archivos.

Cada vez que se confirma o se guarda el estado de un proyecto en Git, básicamente se toma una fotografía de cómo se ven todos los archivos en ese momento y se almacena una referencia a esa instantánea.



INSTALAR GIT

PARA WINDOWS SE DEBE INGRESAR A :

[HTTPS://GITFORWINDOWS.ORG/](https://gitforwindows.org/)

PARA MAC:

[HTTPS://GIT-SCM.COM/DOWNLOAD/MAC](https://git-scm.com/download/mac)

O DIRECTAMENTE ACCEDIENDO A:

[HTTPS://GIT-SCM.COM/DOWNLOADS](https://git-scm.com/downloads)

Downloads



macOS

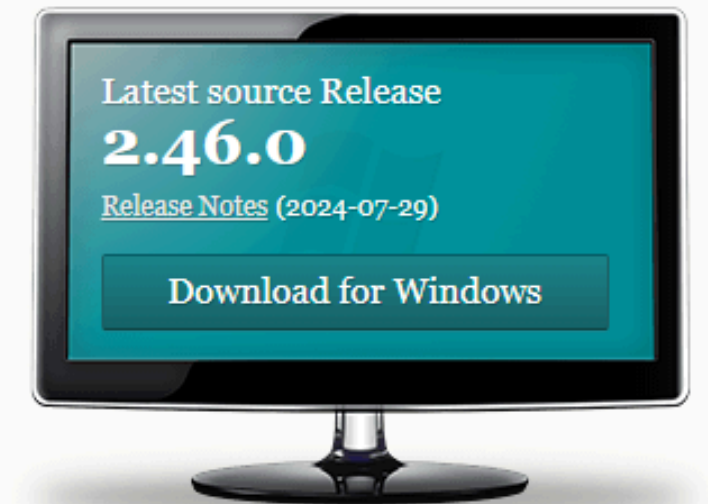


Windows



Linux/Unix

Older releases are available and the [Git source repository](#) is on GitHub.



GUI Clients

Git comes with built-in GUI tools ([git-gui](#), [gitk](#)), but there are several third-party tools for users looking for a platform-specific experience.

[View GUI Clients →](#)

Logos

Various Git logos in PNG (bitmap) and EPS (vector) formats are available for use in online and print projects.

[View Logos →](#)

Git via Git

If you already have Git installed, you can get the latest development version via Git itself:

```
git clone https://github.com/git/git
```

You can also always browse the current contents of the git repository using the [web interface](#).

COMANDOS

EL PRIMER PASO ES **CONFIGURAR NUESTRO MAIL Y NUESTRO NOMBRE DE USUARIO** PARA QUE CADA VEZ QUE HAGAMOS UN CAMBIO PUEDA REGISTRAR ESOS VALORES PARA INDICAR QUIEN LO HIZO

git config --list --show-origin
git config --list



NOS PERMITE LISTAR TODAS
LAS CONFIGURACIONES
BÁSICAS

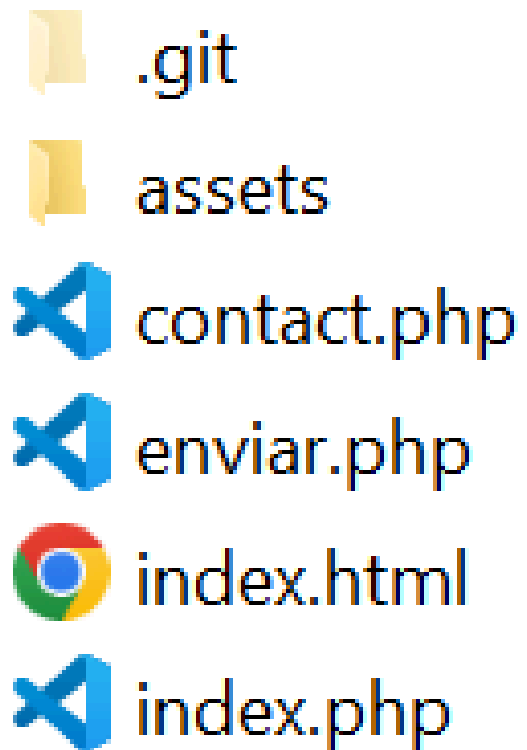
git config --global user.name "Nombre de usuario"
git config --global user.email "email"



PERMITE
MODIFICAR
NOMBRE Y EMAIL

REPOSITARIOS

UN REPOSITORIO EN GIT ES LA CARPETA **.GIT/** DENTRO DE UN PROYECTO. ESTA CARPETA LA CREA GIT AL INICIALIZAR EL PROYECTO



- .git
- assets
- contact.php
- enviar.php
- index.html
- index.php

UN DIRECTORIO **.GIT** TIENE LA SIGUIENTE ESTRUCTURA

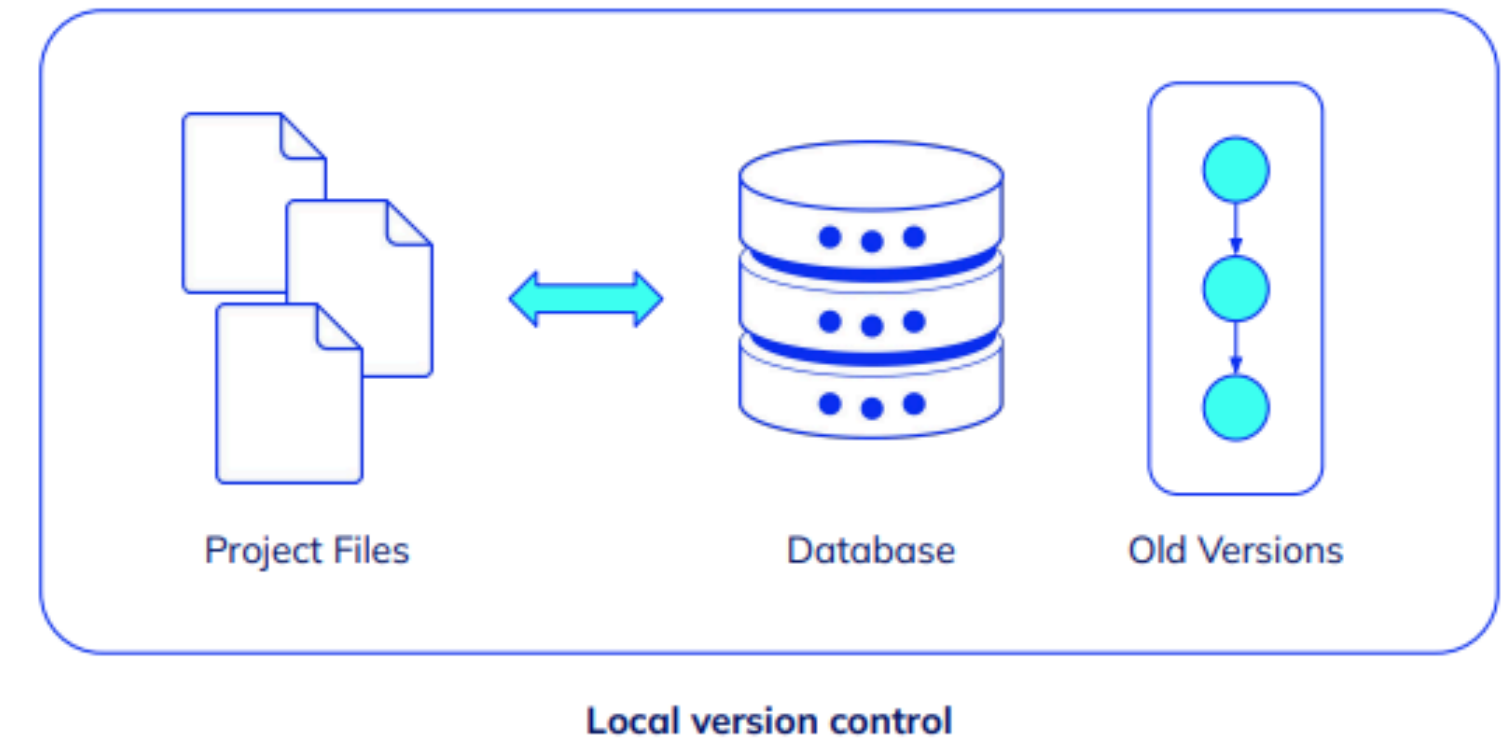
- hooks
- info
- logs
- objects
- refs
- COMMIT_EDITMSG
- config
- description
- HEAD
- index

Rastrea todos los cambios realizados en los archivos de un proyecto y crea un historial a lo largo del tiempo. **Si se elimina la carpeta .git/ entonces desaparece el historial del proyecto**

TIPOS DE REPOSITORIOS

REPOSITORIOS LOCALES

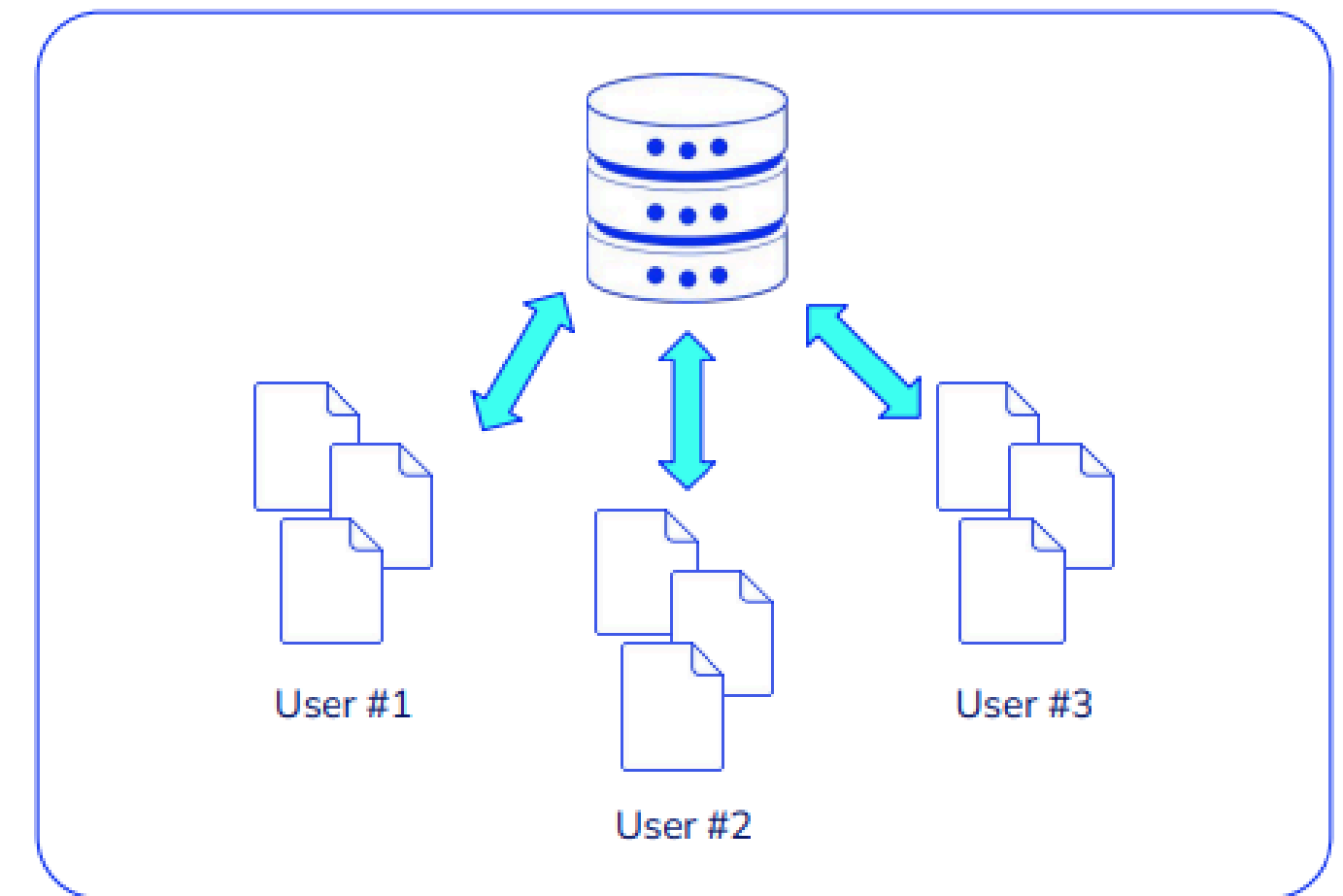
- **EXISTE EN EL SISTEMA DE ARCHIVOS DE TU COMPUTADORA**
- ESTÁ COMPUESTO POR TODOS LOS ARCHIVOS DEL PROYECTO Y UN SUBDIRECTORIO OCULTO **.GIT** QUE CONTIENE TODA LA INFORMACIÓN DEL HISTORIAL DE CAMBIOS, CONFIGURACIONES, Y METADATOS NECESARIOS PARA QUE GIT FUNCIONE.
- **NO OFRECE LA POSIBILIDAD DE COMPARTIR EL CÓDIGO EN CASO DE QUE SE ELIJA TRABAJAR EN MODO DE DESARROLLO COLABORATIVO**



TIPOS DE REPOSITARIOS

REPOSITARIOS CENTRALIZADOS

- EN LUGAR DE ALMACENAR LOS CAMBIOS Y VERSIONES EN EL DISCO DURO, PASARON A GUARDARLAS EN UN SERVIDOR.
- EL AVANCE FRENTE A LOS SISTEMAS DE CONTROL DE VERSIONES LOCALES FUE ENORME, **LOS SISTEMAS CENTRALIZADOS TRAJERON NUEVOS RETOS COMO MÚLTIPLES USUARIOS TRABAJANDO EN UN MISMO ARCHIVO AL MISMO TIEMPO.**
- SI SE CORTABA LA CONEXIÓN A INTERNET O AL SERVIDOR NO SE PODÍA SEGUIR TRABAJANDO.

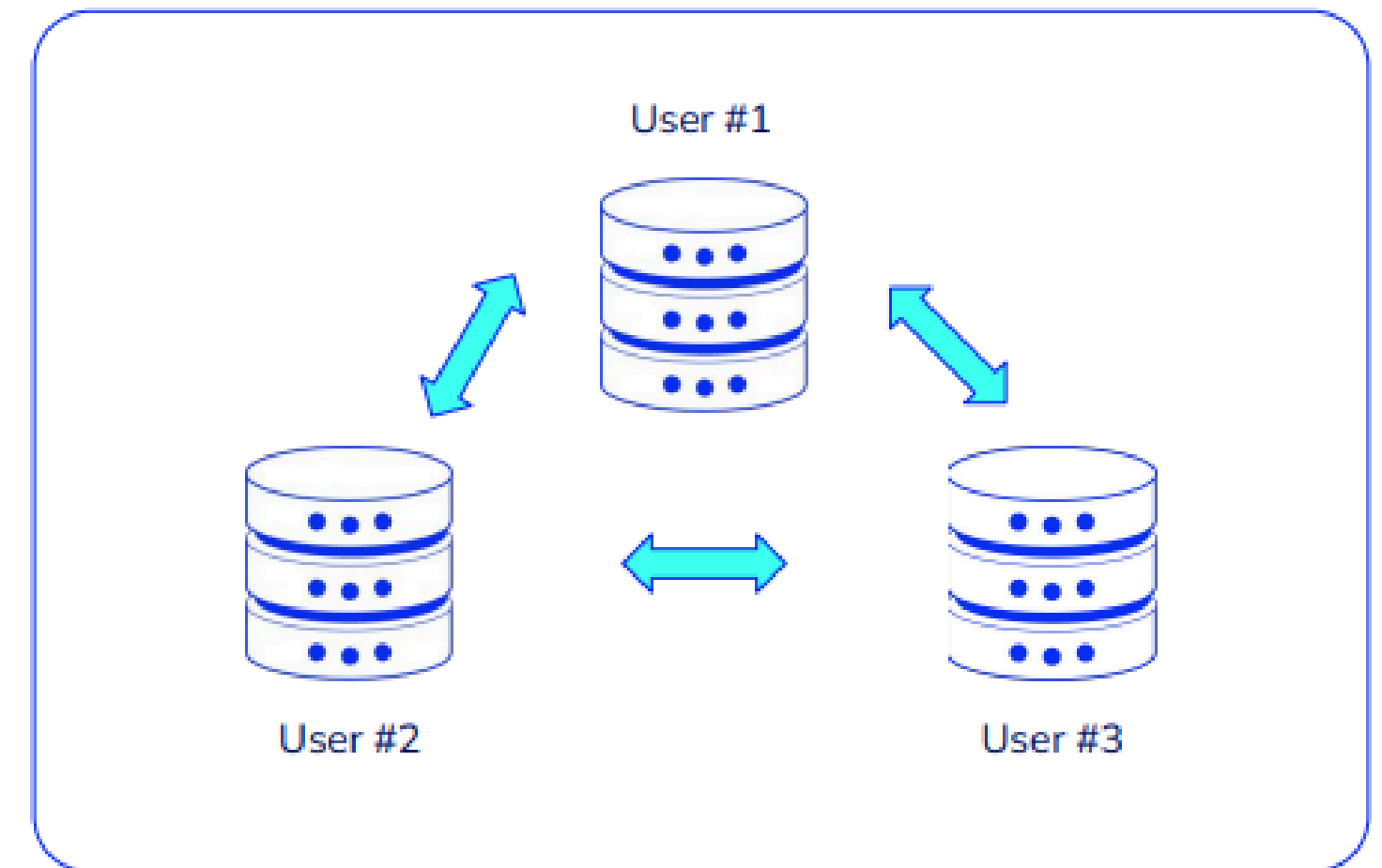


Centralized version control

TIPOS DE REPOSITARIOS

REPOSITARIOS DISTRIBUIDOS

- **CADA DESARROLLADOR TIENE UNA COPIA LOCAL DE TODO EL PROYECTO.**
- DE ESTA MANERA SE CONSTRUYÓ UNA RED DISTRIBUIDA DE REPOSITARIOS, EN LA QUE CADA DESARROLLADOR PODÍA TRABAJAR DE MANERA AISLADA PERO TENIENDO UN MECANISMO DE RESOLUCIÓN DE CONFLICTOS MUCHO MÁS ELEGANTE QUE EN SU VERSIÓN ANTERIOR



Distributed version control

TIPOS DE REPOSITARIOS

STAGED

SIGNIFICA QUE SE HA MARCADO UN ARCHIVO MODIFICADO EN SU VERSIÓN ACTUAL PARA PASAR A SU PRÓXIMO SNAPSHOT DE CONFIRMACIÓN

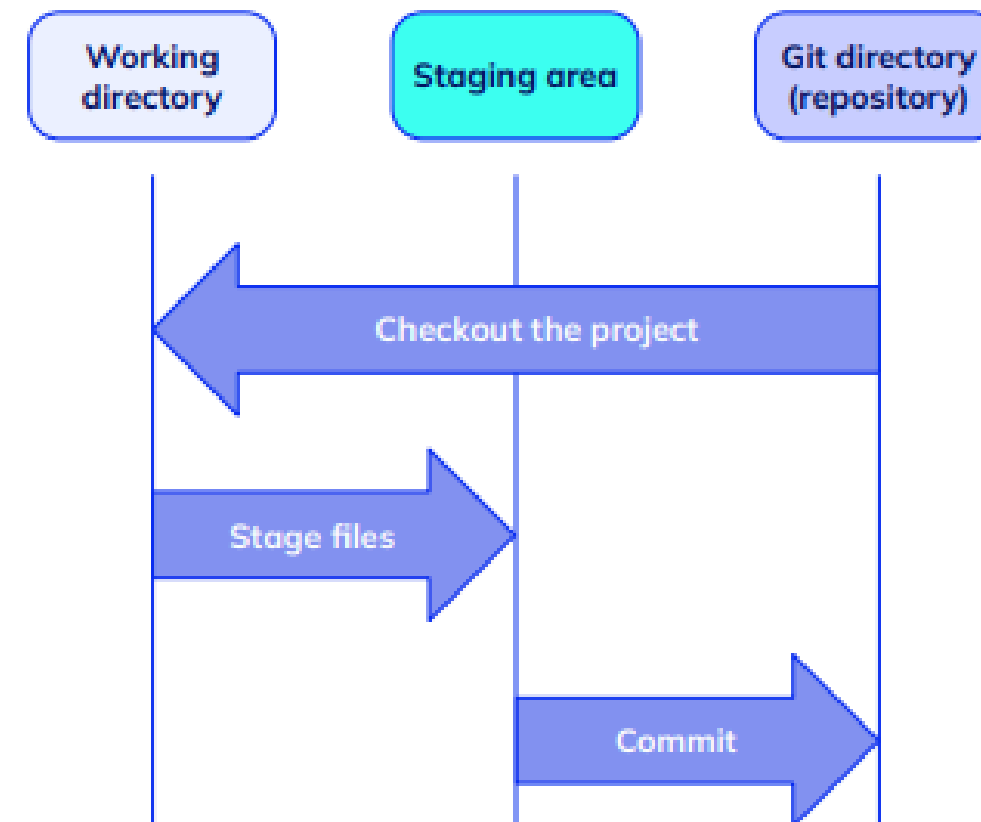
MODIFIED

SIGNIFICA QUE SE HA MARCADO UN ARCHIVO MODIFICADO EN SU VERSIÓN ACTUAL PARA PASAR A SU PRÓXIMO SNAPSHOT DE CONFIRMACIÓN: **SIGNIFICA QUE SE HA CAMBIADO EL ARCHIVO PERO AÚN NO SE HA CONFIRMADO EN SU BASE DE DATOS**

COMMITTED

SIGNIFICA QUE LOS DATOS SE ALMACENAN DE FORMA SEGURA EN SU BASE DE DATOS

Local Operations



INICIALIZAR PROYECTO

SE PUEDE OBTENER UN REPOSITORIO GIT DE DOS MANERAS:

- CONVERTIR UN DIRECTORIO LOCAL QUE, ACTUALMENTE, NO ESTA BAJO NINGÚN CONTROL DE VERSIÓN A UN REPOSITORIO DE GIT
- CLONAR UN REPOSITORIO DE GIT EXISTENTE DESDE ALGÚN OTRO LUGAR

git init



En el caso de tener un directorio de proyecto, que actualmente no esta bajo ningún control de versión, y **desea comenzar a controlarlo con Git, debe seguir los siguientes pasos:**

1. Dirigirse al directorio que se quiere convertir en repositorio
2. Una vez en el repositorio, ejecutar **git init**
3. Esto va a crear un nuevo subdirectorio llamado **.git** que contiene todos los archivos necesarios para el repositorio - el esqueleto de un repositorio git

INICIALIZAR PROYECTO

```
julia@DESKTOP-6D0HCII MINGW64 /d/Organizador/UAE/Segundo cuatrimestre/Programación 1/Curso Monserrat/Prueba Rep  
ositorio
```

```
• $ git init  
Initialized empty Git repository in D:/Organizador/UAE/Segundo cuatrimestre/Programación 1/Curso Monserrat/Prueba Repositorio/.git/
```

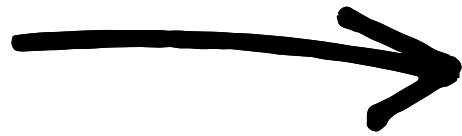
Luego con el comando **-ls -al** se listan los directorios creados:

- **ls:** lista los archivos
- **-a:** muestra los archivos ocultos
- **-l:** muestra la lista en formato detallado

```
julia@DESKTOP-6D0HCII MINGW64 /d/Organizador/UAE/Segundo cuatrimestre/Programación 1/Curso Monserrat/Prueba Repositorio (master)  
$ ls -al  
total 0  
• drwxr-xr-x 1 julia 197609 0 ago. 11 10:01 ./  
drwxr-xr-x 1 julia 197609 0 ago. 11 10:01 ../  
drwxr-xr-x 1 julia 197609 0 ago. 11 10:01 .git/
```

COMANDOS

git status



COMANDO QUE MUESTRA EL ESTADO ACTUAL DE TU REPOSITORIO.

PROPORCIONA LA SIGUIENTE INFORMACIÓN:

- **INDICA LA RAMA EN QUE ESTAS TRABAJANDO**
- **CAMBIOS NO PREPARADOS:** MUESTRA LOS ARCHIVOS QUE HAN SIDO MODIFICADOS EN EL DIRECTORIO LOCAL, PERO AÚN NO HAN SIDO AÑADIDOS A LA “STAGING AREA”
- **CAMBIOS PREPARADOS:** MUESTRA LOS ARCHIVOS QUE HAN SIDO AÑADIDOS A LA “STAGING AREA” Y ESTAN LISTOS PARA SER COMMITADOS
- **ARCHIVOS NO RASTREADOS:** LISTA DE LOS ARCHIVOS NUEVOS QUE GIT AUN NO ESTA SIGUIENDO

```
• $ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```


COMANDOS

Creamos un nuevo archivo dentro del proyecto, por ejemplo “**info.txt**” y luego volvemos a ejecutar **git status**

```
● $ git status
○ On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    info.txt

nothing added to commit but untracked files present (use "git add" to track)
```

El archivo **info.txt** no tiene seguimiento ya que se encuentra debajo de **Untracked files**

Untracked significa que git ve un archivo que no tenía en la versión anterior del proyecto (**snapshot**)

GIT NO LO VA A INCLUIR HASTA QUE NO SE INDIQUEMOS EXPLICITAMENTE

COMANDOS

git add



PARA INCLUIR Y DARLE SEGUIMIENTO A UN ARCHIVO SE PUEDE USAR EL COMANDO:

```
• $ git add info.txt
```

Si volvemos a ejecutar **git status** veremos que nuestro archivo esta bajo seguimiento y listo para ser confirmado, es decir, formar parte de la nueva versión del repositorio (**commit**).

```
julia@DESKTOP-6D0HCII MINGW64 /d/Organizador/UAE/Segundo cuatrimestre/Programación 1/Curso Monserrat/Prueba
Repositorio (master)
• $ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   info.txt
```

COMANDOS

git add



EL COMANDO GIT ADD PUEDE ACEPTAR EL NOMBRE DE:

- **ARCHIVO**
- **DIRECTORIO**

SI ES UN DIRECTORIO, EL COMANDO AGREGA TODOS LOS ARCHIVOS EN ESE DIRECTORIO

SI TENGO MÁS DE UN ARCHIVO QUE QUIERO AGREGAR, PUEDO EJECUTAR **GIT ADD**.

COMANDOS

git add



SI LUEGO DE HACER EL GIT ADD INFO.TXT MODIFICO EL ARCHIVO Y VUELVO A REALIZAR UN GIT STATUS SE VA A VISUALIZAR LA SIGUIENTE INFORMACIÓN:

```
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   archivo1.txt
    new file:   archivo2.txt
    new file:   info.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   info.txt
```

El archivo **info.txt** aparece en la sección **Changes to be committed** y dentro de **Changes not staged for commit**.

Si hacemos un commit en este momento los últimos cambios sobre el archivo no se van a agregar.

COMANDOS

git commit



git commit -m "Comentario"

Luego de la opción -m se debe agregar el comentario

AHORA QUE EL ARCHIVO SE ENCUENTRA EN EL ÁREA DE PREPARACIÓN (STAGE AREA) ESTAMOS LISTOS PARA CONFIRMAR LOS CAMBIOS, ES DECIR REALIZAR UN COMMIT.

PODEMOS EJECUTARLO CON:

GIT COMMIT

AL HACER ESTO SE VISUALIZARÁ EN EL EDITOR ELEGIDO EL SIGUIENTE MENSAJE:

```
julia@DESKTOP-6D0HCII MINGW64 /d/Organizador
ositorio (master)
● $ git commit -m "Ultimos cambios"
[master 9ddd3b8] Ultimos cambios
1 file changed, 1 insertion(+)
```

La salida del comando nos informa sobre que rama lo ejecuto (**master**), el identificador del commit **9ddd3b8** y la cantidad de archivos modificados, y la estadística sobre las líneas que fueron insertadas o eliminadas

MALA PRACTICA



NO COLOCAR MENSAJES EN LOS COMMIT

BUENA PRACTICA



**COLOCAR COMENTARIOS EN LOS COMMIT QUE DESCRIBAN
QUE SE AGREGA O MODIFICA**

Atajo

Si queremos realizar el **add** y el **commit** con un solo comando lo podemos hacer de la siguiente forma:

git commit -a -m “Comentario”

De esta manera se puede omitir el comando
git add

**FUNCIONA SIEMPRE QUE LOS ARCHIVOS
NO SEAN NUEVOS.
ES DECIR HACER ADD DE
MODIFICACIONES**



git log

Con el comando **git log** podemos ver hacia atrás todo lo que ha sucedido en el repositorio.

Muestra el más reciente primero.

```
PS D:\Organizador\UADE\Segundo cuatrimestre\Programación 1\Curso Monserrat\Clase 2\Prueba> git log
● commit 17a6854589eeebd9ae296c3cef4648d258c3c7a8 (HEAD -> master)
  Author: BEETS <beetsftb@gmail.com>
  Date:   Sun Aug 11 18:02:54 2024 -0300

    Segundo commit

commit d02683bdf39c38f150db9bf9068bba2faebe71c7
  Author: BEETS <beetsftb@gmail.com>
  Date:   Sun Aug 11 17:53:07 2024 -0300

    Agrego archivo
```

git log

git log --oneline muestra la información más abreviada de cada commit

```
PS D:\Organizador\UADE\Segundo cuatrimestre\Programación 1\Curso Monserrat\Clase 2\Prueba> git log --oneline
17a6854 (HEAD -> master) Segundo commit
● d02683b Agrego archivo
```

git log nombre_archivo para ver el log de un archivo en particular

git show

Muestra los detalles completos de un solo commit, o si no se envían parámetros muestra la información del último commit

```
PS D:\Organizador\UADE\Segundo cuatrimestre\Programación 1\Curso Monserrat\Clase 2\Prueba> git show info.txt
```

```
● commit ce9f62ad948012a80b7e8e6cc689f70b434c112b (HEAD -> master)
```

```
Author: BEETS <beetsftb@gmail.com>
```

```
Date:   Sun Aug 11 18:25:45 2024 -0300
```

```
    ultimo commit
```

```
diff --git a/info.txt b/info.txt
```

```
index a5eedee..f778431 100644
```

```
--- a/info.txt
```

```
+++ b/info.txt
```

```
@@ -1,2 +1,4 @@
```

```
    Prueba prueba!
```

```
-prueba prueba
```

```
\ No newline at end of file
```

```
+prueba prueba
```

```
+
```

```
+aca cambie!
```

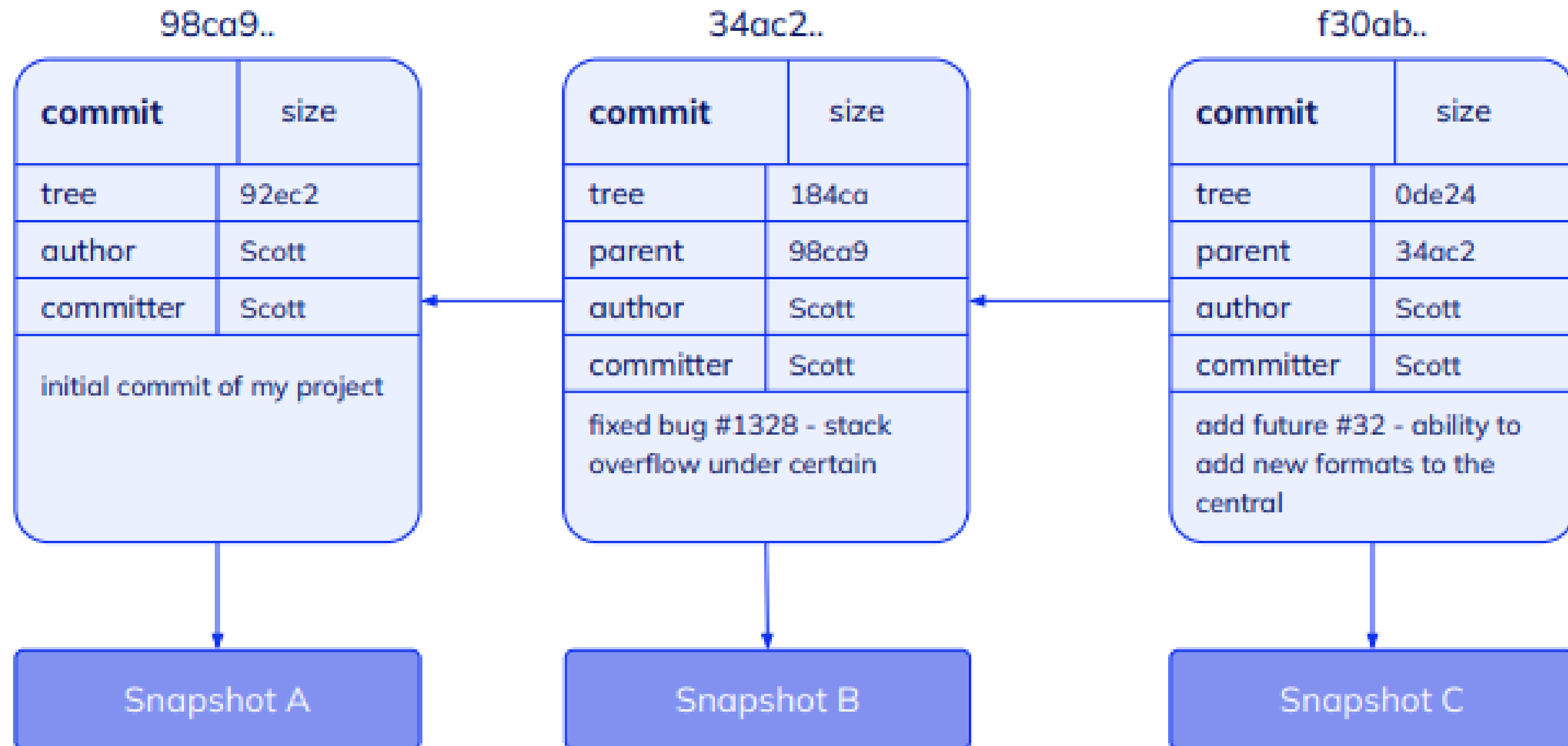
```
\ No newline at end of file
```

En **diff** muestra las diferencias de los cambios realizados en el commit

Ramas o Branches

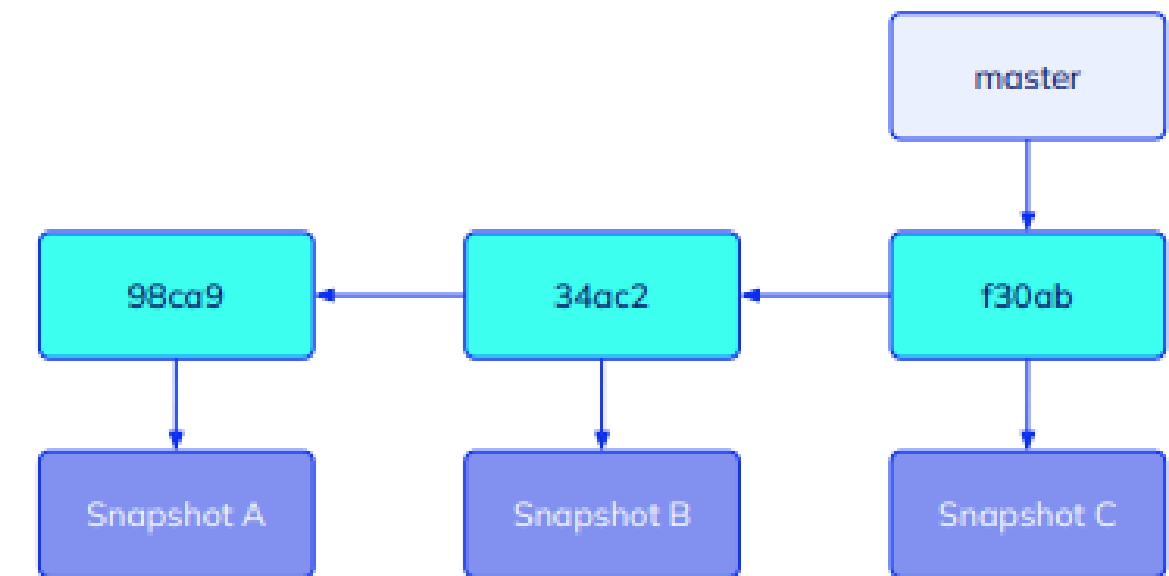
- **Git** almacena como una serie de snapshots (copias puntuales de los archivos completos, tal y como se encuentran en ese momento).
- En cada confirmación de cambios (commit), Git almacena un punto de control que conserva:
 - Un apuntador a la copia puntual de los contenidos preparados (staged).
 - Metadatos con el autor y el mensaje explicativo.
 - Uno o varios apuntadores a las confirmaciones (commit) que sean padres directos de esta.

Ramas o Branches



Branch

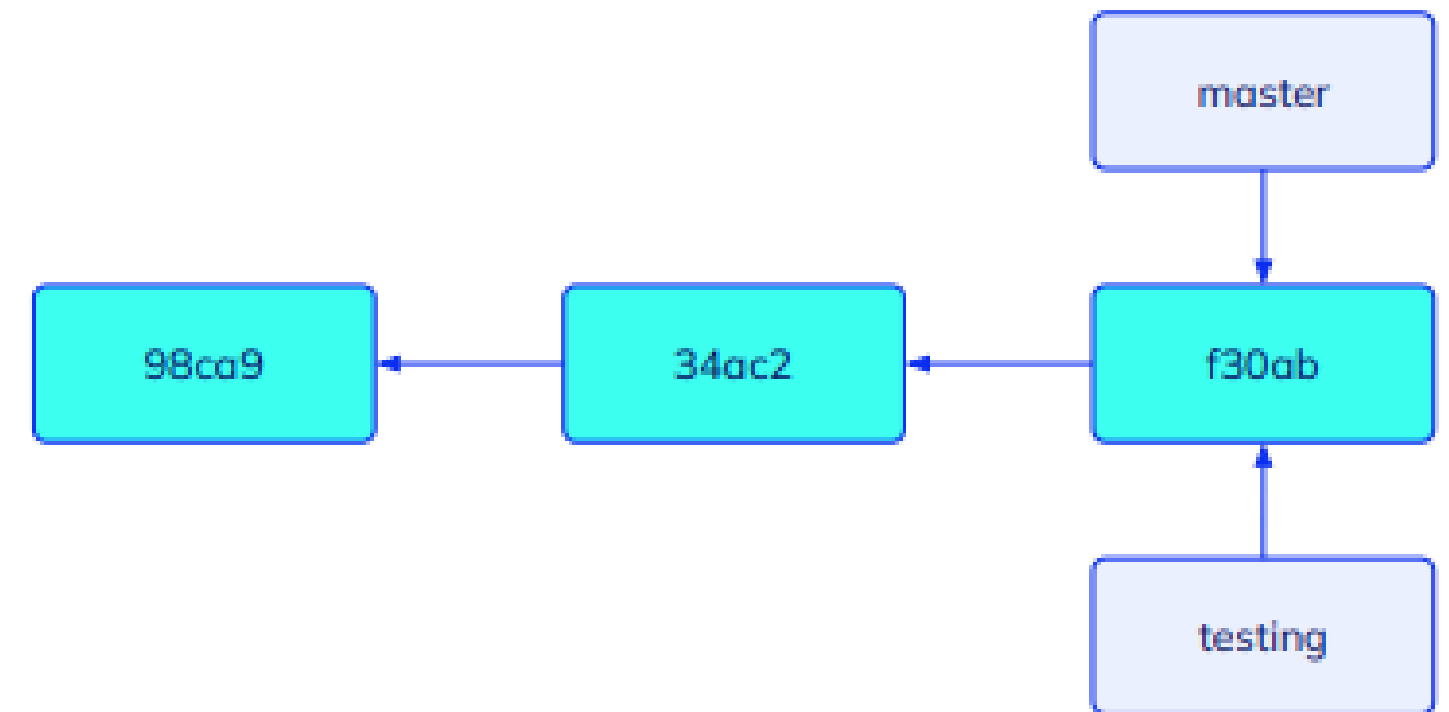
- Un **branch** o una **rama** es un **apuntador movil** que apunta a una de esas confirmaciones o commit.
- **La rama por defecto de Git es master o main**
- En cada commit de cambios que hagamos, la rama ira avanzando automáticamente.
- **La rama master apuntará siempre a la última confirmación realizada**



Branch

- Cuando se crea una rama o un branch, **se crea un nuevo apuntador**
- Es conveniente pensarlo como una “**bifurcación**” o “**copia del código**” en la que podemos trabajar sin afectar otras ramas.
- Para crear una nueva rama se utiliza el comando

git branch nombre_branch



Esto creará un nuevo apuntador a la misma confirmación en la que estamos. ES IMPORTANTE DESTACAR QUE CREA LA RAMA, PERO NO SALTA A LA MISMA

Branch

```
PS D:\Organizador\UADE\Segundo cuatrimestre\Programación 1\Curso Monserrat\Clase 2\Prueba> git branch develop
```

Este comando crea una nueva rama, a partir de **master** (porque estamos en master), y como nombre le pone **develop**

¿Como sabe Git en que rama estas en este momento?

- Mediante un **apuntador especial denominado HEAD**
- **HEAD** es un apuntador a la rama local en la que estamos en un momento.

git checkout

- Para **saltar de una rama a otra**, utilizamos el comando **git checkout**
- **Git checkout** mueve el apuntador **HEAD** a la rama que deseamos dirigirnos

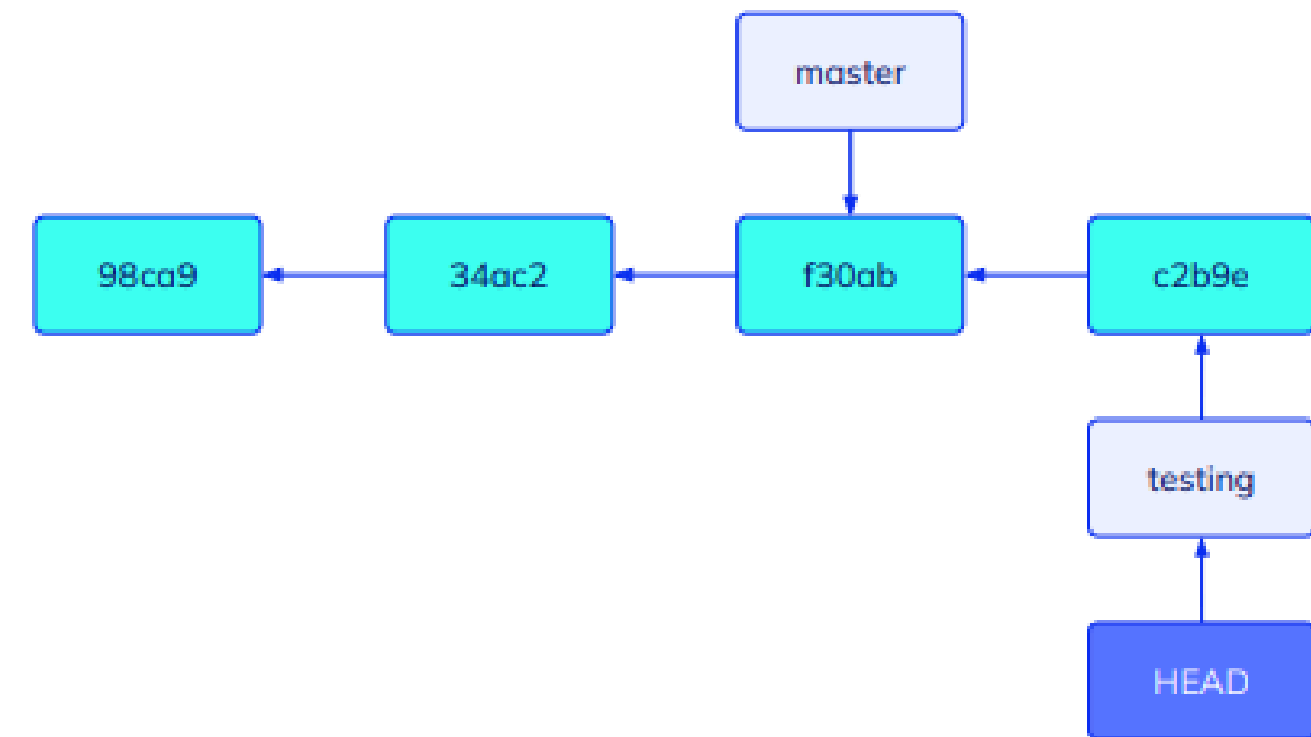
git checkout nombre_rama

```
PS D:\Organizador\UADE\Segundo cuatrimestre\Programación 1\Curso Monserrat\Clase 2\Prueba> git checkout develop  
Switched to branch 'develop'
```

En este momento estamos dentro de la rama **develop**. Es decir todo lo que se desarrollará no afectará a **master** u otra rama.

git checkout

- Si creamos un archivo en la rama **develop** , subimos los cambios.
- Luego nos volvemos a la rama **master**, vamos a ver que el último archivo no va a estar.
- Basicamente lo que va a suceder es que la rama, en la imagen de abajo “**testing**” va a estar más delante de **master**



git checkout

- **git checkout -b <nombre_rama>** permite crear la rama y además saltar a la nueva rama de forma inmediata

Integrar cambios

- Una vez que el código de un branch ya esta estable, en un ambiente real pasa las pruebas **vamos a tener que integrarlos con la rama master**
- Para esto podemos utilizar el comando **git merge** desde el destino donde queremos traer los cambios de otra rama.
- Por ejemplo si a **master** queremos traer los cambios de **Testing**, desde la rama master vamos a ejecutar **git merge Testing**

Integrar cambios

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UAE/Programacion 1/Clase 2/Prueba repositorio (master)
● $ git merge Testing
Updating 24e00c4..c576f74
Fast-forward
 cambios.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cambios.txt
```

- Si lo que intentamos fusionar es una rama que esta mas adelantada, va a realizarlo sin problema y se visualizara el mensaje “Fast-forward”

Listar branch

- Para listar las ramas disponibles o los branch que tenemos se puede ejecutar el comando
 - **git branch --list**

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UADE/Programacion 1/Clase 2/Prueba repositorio (master)
● $ git branch --list
    Testing
* master
```

- En color verde se visualiza el branch en el que nos encontramos

Resolución de conflictos

- Cuando modifique en diferentes ramas los mismos archivos. Al hacer **git merge** se visualizará que existen conflictos.
- Se visualizará el siguiente error:

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UAE/Programacion 1/Clase 2/Prueba repositorio (master)
⊗ $ git merge Testing
Auto-merging archivo1.txt
CONFLICT (content): Merge conflict in archivo1.txt
Automatic merge failed; fix conflicts and then commit the result.
```

- Si ejecutamos el comando **git status** nos listará bajo **unmerged** todo aquello conflictivo.

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UAE/Programacion 1/Clase 2/Prueba repositorio (master|MERGING)
● $ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   archivo1.txt

no changes added to commit (use "git add" and/or "git commit -a")
```


Resolución de conflictos

- Además git añade unos marcadores especiales para indicar las diferencias entre ambas ramas.

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
1 <<<<<< HEAD (Current Change)
2 Tambien la modifique en master
3 =====
4 Modificacion 1 en testing
5 >>>>>> Testing (Incoming Change)
6
```

- Para resolverlo, se debe elegir manualmente con que versión nos queremos quedar y luego realizar un git commit.
- Es importante destacar que si elegimos quedarnos con la versión de master, esos cambios no van a quedar en el branch.

Eliminar un branch

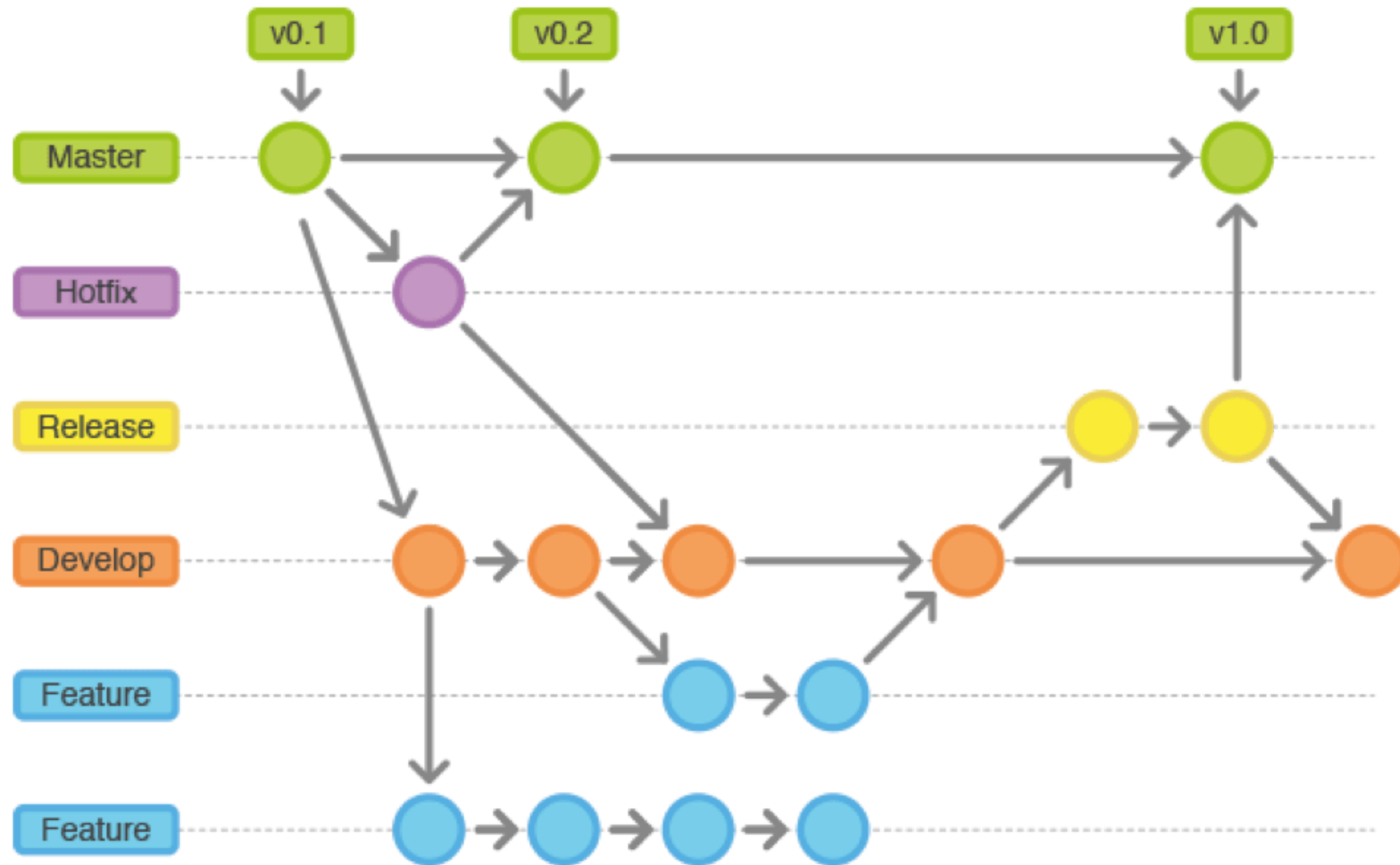
- Para eliminar un branch se tiene que utilizar el comando **git branch -d <nombre_rama>**

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UAE/Programacion 1/Clase 2/Prueba repositorio (master)
● $ git branch -d "otro_branch"
Deleted branch otro_branch (was 6779317).
```

GitFlow

- Es un flujo de trabajo de Git que nos permite **gestionar ramas e implementaciones en un proyecto**
- Fue popularizado por Vincent Driessen en 2010
- Hoy en día, la mayoría de los equipos de desarrollo la utilizan como una estrategia estructurada para gestionar versiones y colaboraciones en proyectos

GitFlow



GitHub

- Es una plataforma de desarrollo colaborativo que permite a los desarrolladores **alojar, gestionar, y compartir proyectos de software.**
- Es popular para proyectos que utilizan Git
- GitHub se utiliza para gestionar el historial de versiones de un proyecto y facilitar la colaboración entre múltiples desarrolladores.

GitHub - Crear una cuenta gratuita

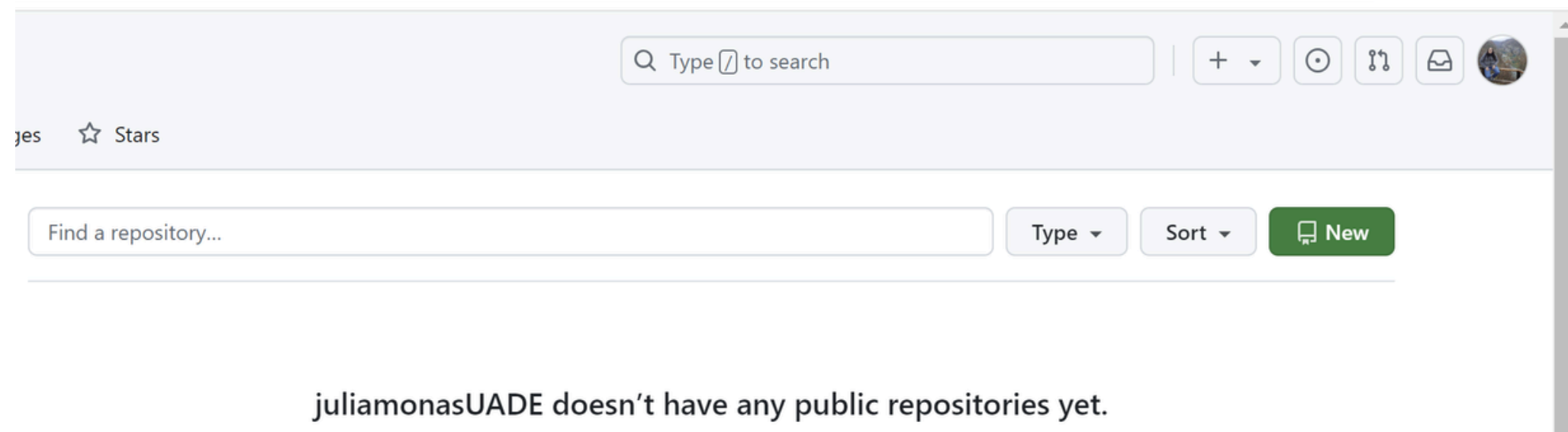
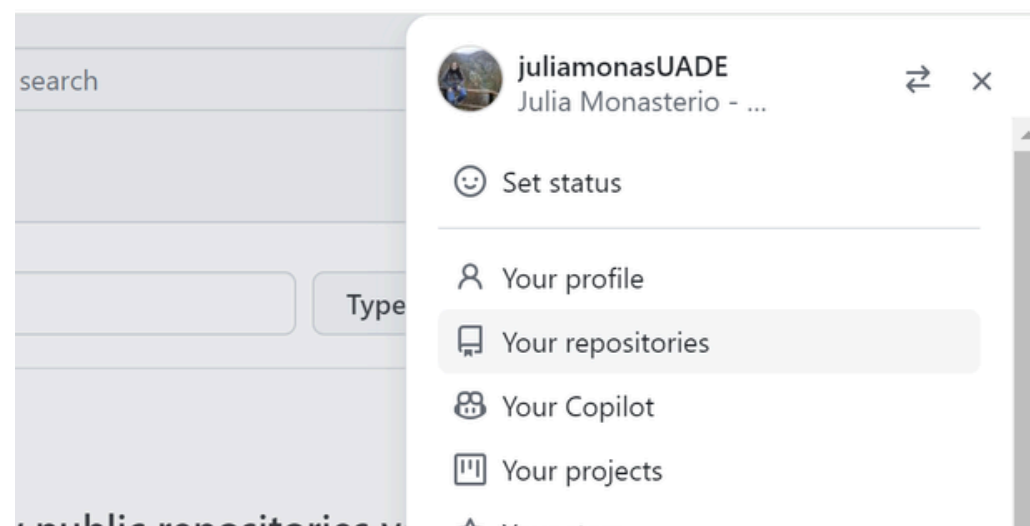
- Ingresar a **github.com** y presionar en **Sign up** para crear una cuenta
- Se deberá ingresar un correo electrónico y luego una password

GitHub - Crear un repositorio remoto

- Un repositorio remoto contiene versiones de nuestros proyectos que estan siendo almacenados en internet.
- Pueden existir repostorios remotos que son solo de **lectura** y otros de **lectura/escritura**

GitHub - Crear un repositorio remoto

- 1) Hacemos click en nuestro Perfil y seleccionamos la opción **Your Repositories**
- 2) Presionamos **New**



GitHub - Crear un repositorio remoto

Esto nos va a llevar a un formulario como el siguiente:

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository](#)

Required fields are marked with an asterisk (*).

Owner *

Repository name *

Owner

- juliamonasUADE

/

Great repository names are short and memorable. Need inspiration? How about [super-duper-umbrella](#) ?

Description (optional)

☒

Public

Anyone on the internet can see this repository. You choose who can commit.

☐

Private

You choose who can see and commit to this repository.

Initialize this repository with:

☐

Add a README file

This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses](#).

You are creating a public repository in your personal account.

Create repository

Vamos a seleccionar:

- Publico
- Colocarle un nombre
- Una descripción opcional
- Marcar Add a Readme file
- **Presionar Create Repository**

GitHub - Crear un repositorio remoto

The screenshot shows the GitHub interface for a newly created repository named 'Programacion1Belgrano' by user 'juliamonasUADE'. The repository is public and currently has no stars or forks. The main content area displays the initial commit, which includes a 'README.md' file. The README content shows the repository name 'Programacion1Belgrano'. The right sidebar contains sections for 'About' (no description), 'Releases' (no releases published), and 'Packages' (no packages published).

Navigation bar: Code (selected), Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings.

Repository header: Programacion1Belgrano (Public) | Pin | Unwatch (1) | Fork (0) | Star (0)

Branches and tags: main (1 Branch), 0 Tags. Search: Go to file. Actions: Add file, Code.

Commit history: juliamonasUADE Initial commit (1b32f32 · 1 minute ago) 1 Commit.

Files: README.md (Initial commit, 1 minute ago).

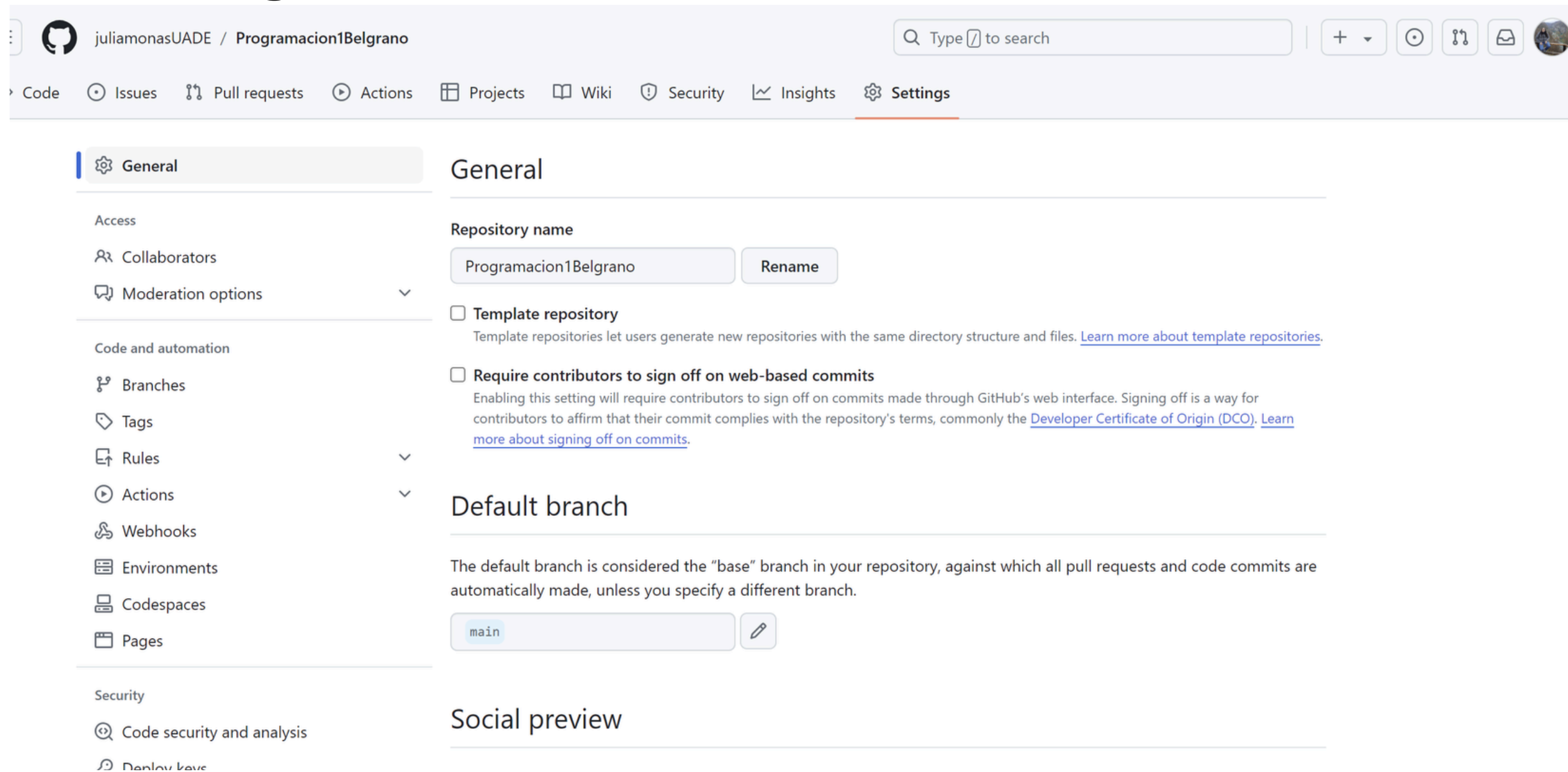
README content: Programacion1Belgrano

Right sidebar:

- About: No description, website, or topics provided.
- Readme
- Activity
- 0 stars
- 1 watching
- 0 forks
- Releases: No releases published. [Create a new release](#)
- Packages: No packages published. [Publish your first package](#)

GitHub - Configuración

Una vez dentro del repositorio, se dirigen a Settings y ahí van a poder modificar la configuración del remoto



GitHub - Configuración

Dentro de Danger Zone van a poder modificar la visibilidad del repositorio, como eliminar el repositorio

Danger Zone

Change repository visibility

This repository is currently public.

[Change visibility](#)

Disable branch protection rules

Disable branch protection rules enforcement and APIs

[Disable branch protection rules](#)

Transfer ownership

Transfer this repository to another user or to an organization where you have the ability to create repositories.

[Transfer](#)

Archive this repository

Mark this repository as archived and read-only.

[Archive this repository](#)

Delete this repository

Once you delete a repository, there is no going back. Please be certain.

[Delete this repository](#)

GitHub - Configuración

Para agregar un nuevo remoto a nuestro repositorio Git, debemos ejecutar el siguiente comando:

git remote add <name> https://github.com/<usuario>/<nombre_del_proyecto>

En name tengo que colocar el nombre con el que quiero conocer al repositorio remoto, **comunmente se coloca origin**

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UAE/Programacion 1/Clase 2/Prueba repositorio (master)  
● $ git remote add origin https://github.com/juliamonasUAE/Programacion1Belgrano.git
```

GitHub - Configuración

Para poder subir al repositorio remoto los cambios locales, debemos utilizar el comando:

git push <remote> <branch>

Este comando va a funcionar solo si:

- Se cuenta con permiso de escritura en ese cliente remoto
- Nadie subió cambios mientras realizábamos e intentábamos subir nuestros cambios

Si alguien más hizo un **push** con anterioridad, la operación será rechazada.

Primero se deben incorporar los cambios nuevos subidos por los demás antes de poder hacer un push

GitHub - Subir nuestros cambios

```
julia@Julia MINGW64 ~/OneDrive/Desktop/Organizador/UAE/Programacion 1/Clase 2/Prueba repositorio (master)
$ git push origin master
info: please complete authentication in your browser...
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 20 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (14/14), 1.17 KiB | 1.17 MiB/s, done.
Total 14 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/juliamonasUADE/Programacion1Belgrano/pull/new/master
remote:
To https://github.com/juliamonasUADE/Programacion1Belgrano.git
 * [new branch]      master -> master
```

 master ▾  2 Branches  0 Tags

  Add file ▾  Code ▾

This branch is 6 commits ahead of, 1 commit behind **main**.

 **beetsftb** Merge branch 'Testing' 59e088b · 12 hours ago  6 Commits

 archivo1.txt	Cambios en master	12 hours ago
 cambios.txt	Incorpore nuevo archivo en testing	12 hours ago
 otro_branch.txt	Otro branch	12 hours ago

GitHub - Obtener cambios

Para poder obtener los cambios de un repositorio remoto se utiliza el comando **git pull**

git pull descargara todos los cambios nuevos que no tenemos en nuestro repositorio local desde el repositorio remoto.

GitHub - git Clone

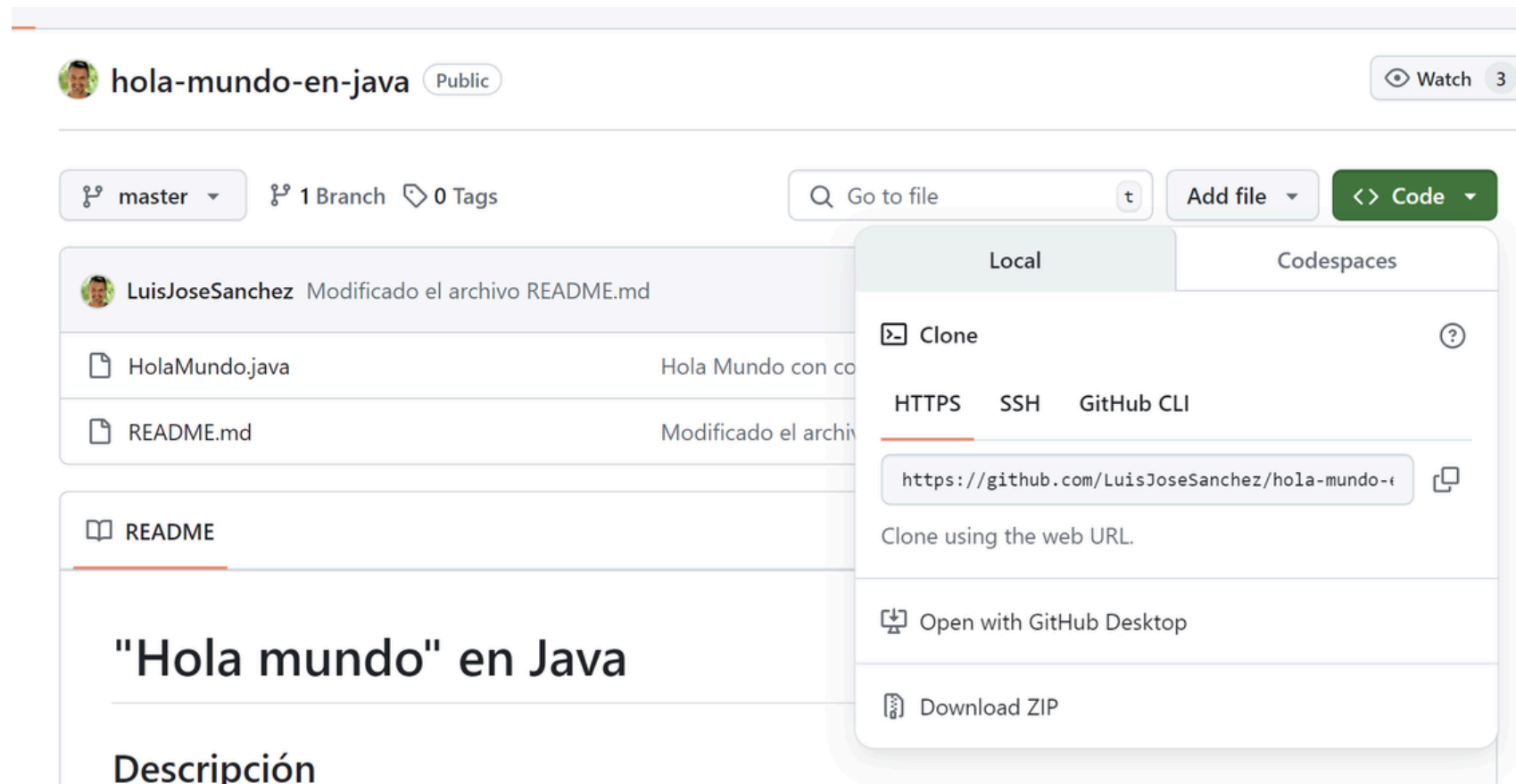
Cuando queremos colaborar en un proyecto que ya esta subido, y que tiene codigo desarrollado, no es necesario crear un repositorio vacio sino debemos obtener una copia del repositorio remoto.

Por ejemplo tomo un repositorio remoto que tiene un hola mundo en java:

<https://github.com/LuisJoseSanchez/hola-mundo-en-java>

GitHub - git Clone

1) Ingreso al repositorio remoto, y copio la url



2) Luego me posiciono en mi local donde quiero que se clone el repositorio y ejecuto **git clone <url>**

GitHub - git Clone

Este comando va a realizar lo siguiente:

- Crea un directorio con el mismo nombre que el repositorio del cliente
- Inicializa un directorio .git dentro de el
- Trae toda la informaci[on de ese repositorio, muestra su ultima version

Si deseo colocarle un nombre diferente al que tiene en el remoto, puedo ejecutar:

git clone <url> <nombre_directorio_nuevo>

Guía de ejercicios Listas

- Crear un directorio en la máquina local e iniciar un repositorio git en el
- Configurar git con los datos de usuario de manera global para el sistema operativo
- Crear un archivo en ese directorio y darle seguimiento en GIT
- Confirmar el archivo dentro de nuestro historial de commits.
- Crear un nuevo archivo en el mismo directorio, editar el archivo que se creó en el punto anterior con cualquier contenido.
- Verificar el estado actual del repositorio para tener un panorama general de los pasos siguientes que podemos realizar sobre él.
- Incorporar en el stage area los cambios del punto 5 y confirmar los cambios

Guía de ejercicios Listas

- Crear una nueva rama llamada “testing” y mostrar las ramas del repositorio
- Crear una nueva rama llamada navbar y mostrar las ramas del repositorio
- Crear una nueva rama llamada accounting y mostrar las ramas del repositorio
- Todas las ramas nacen de master



GRACIAS

Por su atención

